

Chapter 1

Week 10 — .NET & C# Concepts

ENEL712 Embedded Systems Design — Lecture Week 10

Summary

Wider .NET/C# overview: the .NET platform and CLR, automatic memory management with generational GC, namespaces, value vs reference types, structures and collections, and Windows Forms basics.

Topics Covered

- The .NET framework
- Common Language Runtime
- Automatic memory management & generational GC
- Namespaces and the class library
- C# concepts and data types
- Structures and collections
- Graphical user interfaces

1.1 The .NET Framework

- C# is compiled by Roslyn into Common Intermediate Language (CIL/MSIL) — CPU-agnostic.
- Multiple .NET languages (C#, F#, VB.NET) all compile to the same CIL.
- .NET is cross-platform, open-source, and built for high-performance apps.
- Core components: Runtime, Libraries, Compiler, SDK, app stacks (e.g., ASP.NET Core).
- Features: automatic memory management, type safety, async support.

1.1.1 .NET Framework vs Modern .NET

Edition	Notes
.NET Framework	Windows-only; first released 2002; in maintenance (v4.8.1).
Modern .NET	Cross-platform (Windows/Linux/macOS); fully open-source; actively developed (v6, v8, ...).

1.2 Common Language Runtime

- JIT compiles methods to native code on first call; subsequent calls reuse the cached native code.
- Manages memory, threads, code-safety verification, compilation, and security.
- Eliminates many memory leaks via automatic management.
- Allows multiple high-level languages to share the same CIL.

1.3 Automatic Memory Management & Generational GC

- Reference types live on a contiguous managed heap.
- GC algorithm: assume garbage → trace from application roots → free unreachable objects → compact and update pointers.
- Unsafe code (pointers) is allowed in unsafe methods compiled with `/unsafe`.

1.3.1 Generations

Generation	Role
Gen 0	Newest, smallest segment — most collections happen here.
Gen 1	Buffer for objects that survived Gen 0.
Gen 2	Largest, holds long-lived objects; collected less often, more expensive.

1.4 Namespaces and the Class Library

- Namespaces group related classes and prevent naming conflicts (e.g., `System.IO.Stream`).
- using directive imports a namespace.
- .NET Class Library covers File I/O, collections, databases, and GUIs.

1.5 C# Concepts and Data Types

- Value types (`int`, `bool`, `struct`) live directly on the stack (or inline within a heap object); passing copies the value.
- Reference types (`class`, `string`, `array`) hold a reference on the stack and the data on the managed heap; passing copies the reference.
- Common types: `int` (32), `long` (64), `float` (32), `double` (64), `decimal` (128), `bool`, `string`.
- Parameters: `ref` / `out` / `params` modifiers as in Lecture 19.

- Strings: 16-bit chars; supports concatenation (+), interpolation (\$), regular/verbatim (@)/raw (""") literals.

1.6 Structures and Collections

- Structures: value types stored on the stack; convenient for small data; copy directly when initialised.
- ArrayList: dynamic array.
- Dictionary<K,V>: key-value lookups via hash table.
- Queue (FIFO), Stack (LIFO), Hashtable.
- Generic collections (List<T>, Dictionary<K,V>) avoid boxing/unboxing and provide type safety vs older non-generic collections.

1.7 Graphical User Interfaces

- Windows Forms: event-driven; controls respond to user actions (click, change).
- Controls: Label, Button, TextBox, ComboBox.
- Containers: GroupBox, Panel, TabControl.
- Event handlers respond to messages (e.g., plusB_Click).
- Use try/catch in handlers to gracefully handle invalid input.

1.8 Use Cases & Applications

- Modern .NET (8+) is the right choice for new desktop, web, and microservice projects — cross-platform, fast JIT, and active feature work.
- .NET Framework 4.8 is still common in legacy enterprise systems; understanding both saves headaches when porting.
- JIT and generational GC are why .NET app cold-start is slower than native but warm performance is competitive.
- Generics (List<T>, Dictionary<K,V>) avoid boxing/unboxing — preferred for any new code over the legacy ArrayList/Hashtable.

1.9 Worked Example: Boxing Cost

```
ArrayList legacy = new ArrayList();    // stores object -- boxes ints!
List<int> modern = new List<int>();    // stores int directly

for (int i = 0; i < 1_000_000; i++) legacy.Add(i);    // 1M boxings
for (int i = 0; i < 1_000_000; i++) modern.Add(i);    // no boxing
```

Boxing each int allocates a tiny heap object — the legacy version pressures the GC and runs slower for no benefit.

1.10 Quick GC Mental Model

- Most allocations live and die in Gen 0 — practically free.
- Survivors get promoted to Gen 1, then Gen 2.
- Gen 2 collections are expensive — caused by long-lived caches, statics, and event-handler leaks.
- Use weak references for caches that should not pin objects in memory.

1.11 JIT vs AOT

- Default .NET ships CIL and JIT-compiles per method on first call.
- Ahead-of-Time (AOT) compilation produces native binaries — faster startup, smaller deploys, useful for short-lived CLI tools or mobile apps.
- Trade-off: AOT loses some dynamic features (Reflection.Emit, runtime code generation).

Key Definitions

CIL

Common Intermediate Language — bytecode all .NET languages compile to.

JIT Compilation

Runtime translation of CIL to native machine code; native code is cached.

Managed Heap

CLR-reserved contiguous memory for reference types.

Garbage Collection (GC)

Automatic process freeing unreachable objects on the managed heap.

Value Type

Stored directly in its memory location (e.g., int, struct).

Reference Type

Variable holds an address to data on the managed heap (e.g., class, string).

Boxing/Unboxing

Conversion between value type and reference type; avoided by generic collections.

Namespace

Hierarchical naming system grouping related types (System.IO.Stream).

Event Handler

Function executed in response to a GUI/system message.

NuGet Package

Standard .NET deployment unit containing one or more assemblies plus metadata.

Assembly

Compiled .dll or .exe holding CIL + metadata; loaded by the CLR at runtime.

AOT (Ahead-of-Time) Compilation

Compiling CIL to native machine code at build time instead of runtime.

Weak Reference

Reference that does not prevent the GC from collecting the target; useful for caches.

Sources

- Lecture Week 10.pdf